
Project report for CS291A

Mixed and Augmented Reality

By: Satyam Awasthi and Shubham Talbar

Spell Training in VR



Figure 1 Unity based Spell Training Virtual Reality game, where several Monsters have been summoned by a User

OVERVIEW

The ability to perceive the shape and motion of hands can be a vital component in improving the user experience across a variety of technological domains and platforms. In our project we explore utilization of hand gestures, and create an innovative interaction system based on mid-air sketching and build a VR project to best utilize said system. The project is focused on creating an immersive experience for a user placed in a Unity-based virtual world and can cast spells from their magic wand in order to summon creatures and monsters. They also have the ability to manipulate these virtual creatures using gestures and mid-air sketches. The spells casted (or sketches drawn) by the user are recognized via a OpenCV based image recognition module. The engagement process is assisted via a simple camera (in-built camera in laptops/macs) and hence there is no additional equipment baggage for the user. The Unity-based VR app communicates

with the image recognition module via a web-socket server setup on both instances of the project.



Figure 2 Scene setup and development of the game in Unity version 2020.3.27f1

Technology Specifications

Unity Game Engine 2020.3.27f1 was used for creating our cross-platform real-time 3D project and **Vuforia Engine 10.4 SDK** for AR exploration. The **OpenCV 4.5.5** Computer Vision library was used for image processing on a real-time video feed. The Unity and OpenCV based backend systems work seamlessly on Apple and Intel based systems. Both the modules can be run on MacOS and Windows operating systems. Two frameworks lay the foundation to our image recognition module. First is Google's **MediaPipe Hands** for realtime high-fidelity hand landmark and finger tracking. Second is Google **Tesseract OCR**, formerly developed by HP and now maintained by Google for recognizing hand-drawn digits (numericals 0-9) mid-air by the users actively engaged and immersed in the game.

Unity Game Engine

[Unity](#) is a cross-platform game engine developed by Unity Technologies, first announced and released in June 2005. The engine has since been gradually extended to support a variety of desktop, mobile, console and virtual reality platforms. It is particularly popular for iOS and Android mobile game development. The engine can be used to create three-dimensional (3D) and two-dimensional (2D) games, as well as interactive simulations and other experiences. The engine has been adopted by industries outside video gaming, such as film, automotive, architecture, engineering, construction, and the United States Armed Forces.

OpenCV

[OpenCV \(Open Source Computer Vision Library\)](#) is a library of programming functions mainly aimed at real-time computer vision. Originally developed by Intel, it was later supported by Willow Garage then Itseez (which was later acquired by Intel). The library is cross-platform and free for use under the open-source Apache 2 License. Starting in 2011, OpenCV features GPU acceleration for real-time operations.



Figure 3 Different Technologies used for Spell Training Game development

Vuforia Engine

[Vuforia Engine](#) offers the advanced technology needed for companies to differentiate the way they sell and market. Build engaging 3D product demonstrations, interactive brand campaigns, or create applications for customers to view and personalize products in their own homes. Augmented reality connects people to products through experiences that drive revenue and brand recognition.

MediaPipe Hands

[MediaPipe Hands](#) employs machine learning to infer 21-3D landmarks of a hand from just a single frame. Whereas current state-of-the-art approaches rely primarily on powerful desktop environments for inference, this method achieves real-time performance on a mobile phone, and

even scales to multiple hands. The solution utilizes an ML pipeline consisting of multiple models working together: A palm detection model that operates on the full image and returns an oriented hand bounding box. A hand landmark model that operates on the cropped image region defined by the palm detector and returns high-fidelity 3D hand keypoints.

Tesseract

[Tesseract](#) is an optical character recognition engine for various operating systems. It is free software, released under the Apache License. Originally developed by Hewlett-Packard as proprietary software in the 1980s, it was released as open source in 2005 and development has been sponsored by Google since 2006.

Design and Development

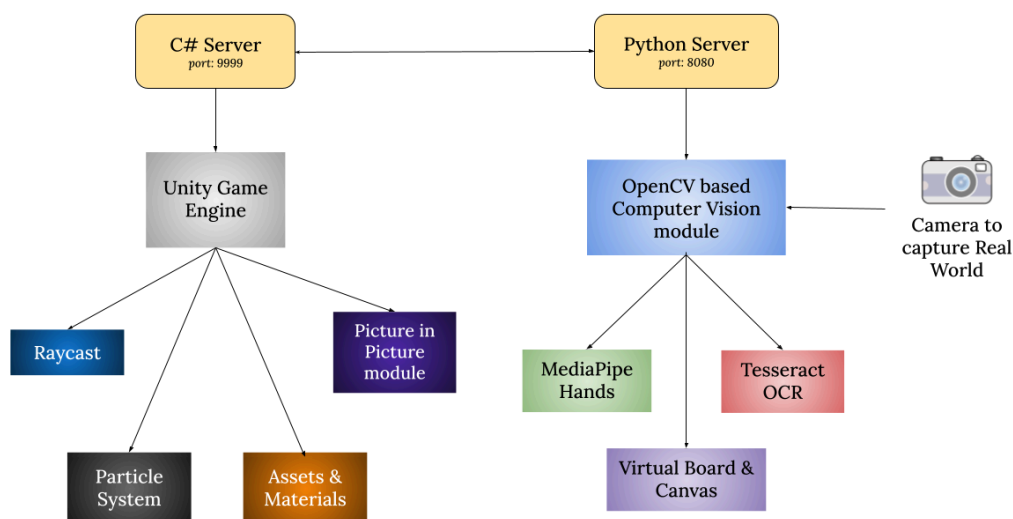


Figure 4 Architecture of the Game developed

The entire project is composed of two backend systems working side-by side - first the Unity Game Engine, and second the Image Recognition module developed using OpenCV, MediaPipe Hands and Tesseract OCR libraries. To establish communication between the two components a server-client protocol was used to send commands/messages. Two servers were set up in total, each in one of the backends. Each backend also had a client connected to the server of the other.

Picture in Picture mode

OpenCV Server in Python: After the game starts, the user can begin to use his wand using the *Fire* button, the C# based (WebSocketSharp) Client sends a message to the OpenCV based backend to initiate hand gesture recognition. Hence, it is essential that this python based socket server is running before the game starts. This step also enables the Picture-In-Picture mode in the game.

Gesture Recognition Module: When the OpenCV based server receives the message to initiate the recognition module and opens a virtual board which serves as a canvas for the user to draw *mid-air* sketches/gestures on. Here the gestures we use are limited to numerical digits (0-9) and not any other specific symbol/sketch.

C# Server in Unity: Since, it would not be seamless for the user to toggle between two different windows (draw on openCV and play on Unity) we decided to send back the user's videofeed overlaid by user's sketching canvas in real time from OpenCV socket client to C# server in Unity. The OpenCV client encodes an image into base64 format and sends to the C# server. The C# server decodes the byte stream received and recreates the sequence of images and renders them in the game's UI component every frame for a Picture-in-Picture framework in Unity. This way the user need not toggle between Unity and OpenCV windows for specifically drawing gestures.

PIP Disabled: After the user-drawn gesture gets recognized, the python server will send its answer to the awaiting client at unity. This will conclude the gesture recognition and will also disable the PIP until another gesture recognition is requested by the unity client.

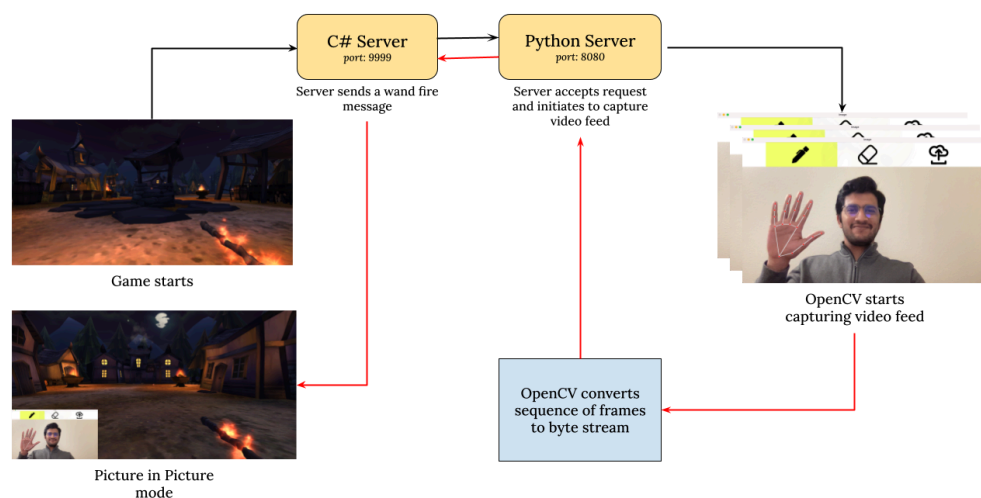


Figure 5 Design for enabling Picture in Picture (PIP) mode in Unity

Image Recognition module

Posture recognition using MediaPipe Hands

Once the OpenCV based server receives a request for hand gesture recognition, it starts capturing the real time video feed of the real world through a camera (typically the same camera inbuilt in a laptop/mac). For recognizing the hand gestures it is crucial that we segment and identify important *landmarks* in the sequence of frames so as to track gestures drawn by the user. For identifying and segmenting the hand in these frames, Google's *MediaPipe Hands* library is used. Identifying the important landmarks empowers the ability to track a specific point/index which serves as the brush for the user to paint on a canvas.

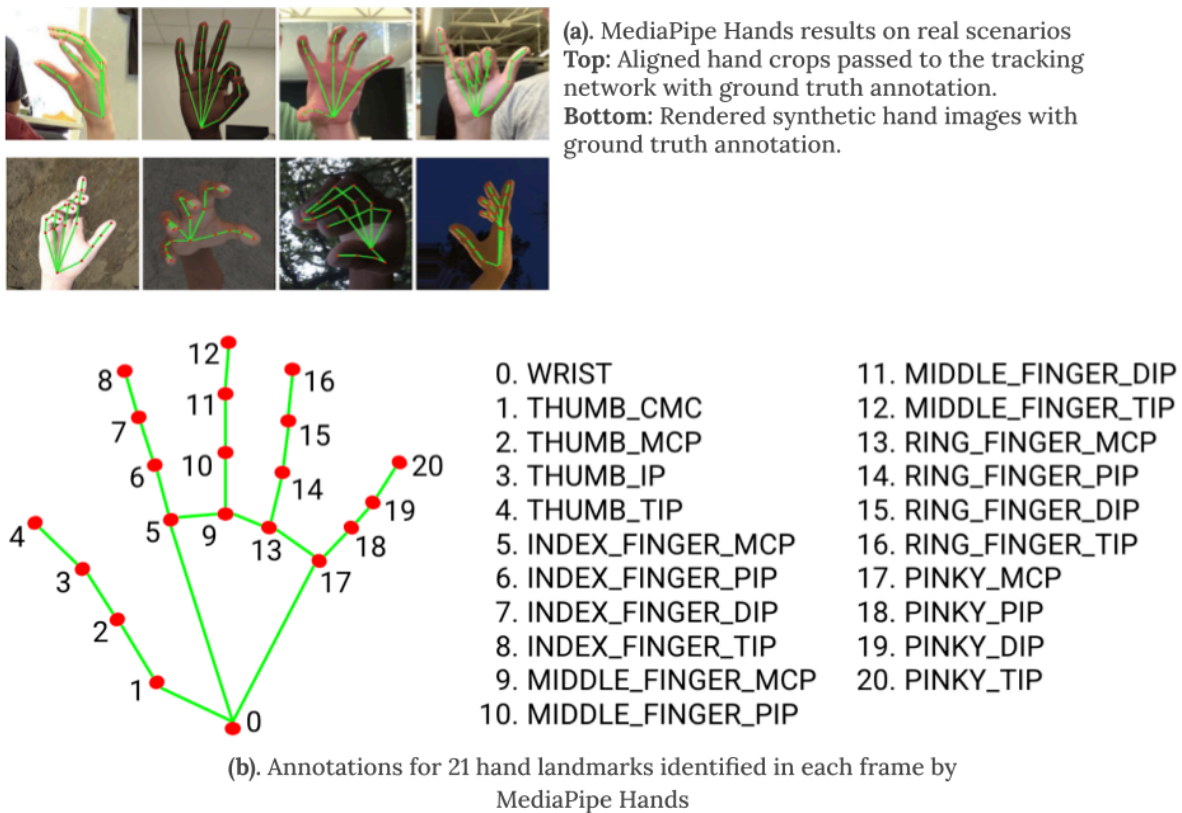


Figure 6 Mediapipe Hands results and annotation results

Based on the location of landmarks relative to each other, certain heuristics could be defined within the scope of the game. For every heuristic rule a certain posture could be defined against which specific triggers could be set up. For the purpose of this game we check whether the fingers in the image are straight-up or not. For this we developed our own heuristic with the code snippet as below-

```
def fingers_up(self):
    fingers = []

    # Thumb
    if self.landmark_list[self.tip_ids[0]][1] < self.landmark_list[self.tip_ids[0] - 1][1]:
        fingers.append(1)
    else:
        fingers.append(0)

    # 4 Fingers
    for index in range(1, 5):
        if self.landmark_list[self.tip_ids[index]][2] < self.landmark_list[self.tip_ids[index] - 2][2]:
            fingers.append(1)
        else:
            fingers.append(0)

    return fingers
```

Figure 7 Heuristic to find whether a finger is straight-up or not

Based on the output of the above heuristic results, the following are the different postures defined:

1. Selection posture
2. Painting posture
3. Clear Canvas posture
4. Recognize gesture posture

The below image shows a screenshot of all the postures-

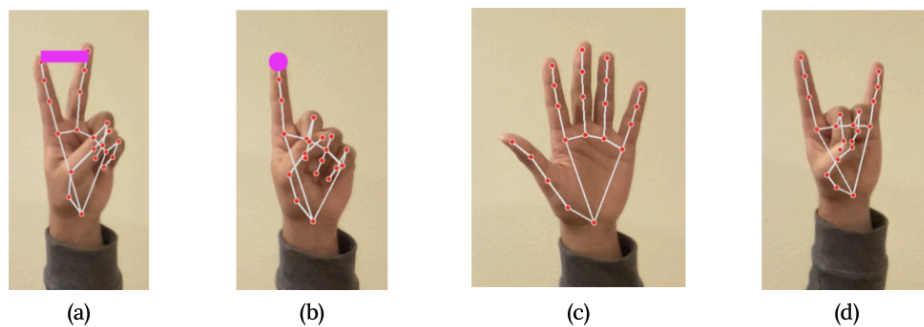


Figure 8 Different heuristic based postures

Freehand Canvas Painting

Once the important landmarks are identified, we can fix a specific landmark (landmark no. 8 in our case) and serve it as a paint brush for the user to draw gestures. The gestures or sketches drawn

by the user are stored and tracked on a separate canvas. This makes the whole gesture tracking and recognition process very simple. The canvas painted by the user is added to the actual videofeed captured by the camera and rendered to the user. This enables the user to understand exactly what they are drawing and make changes accordingly.

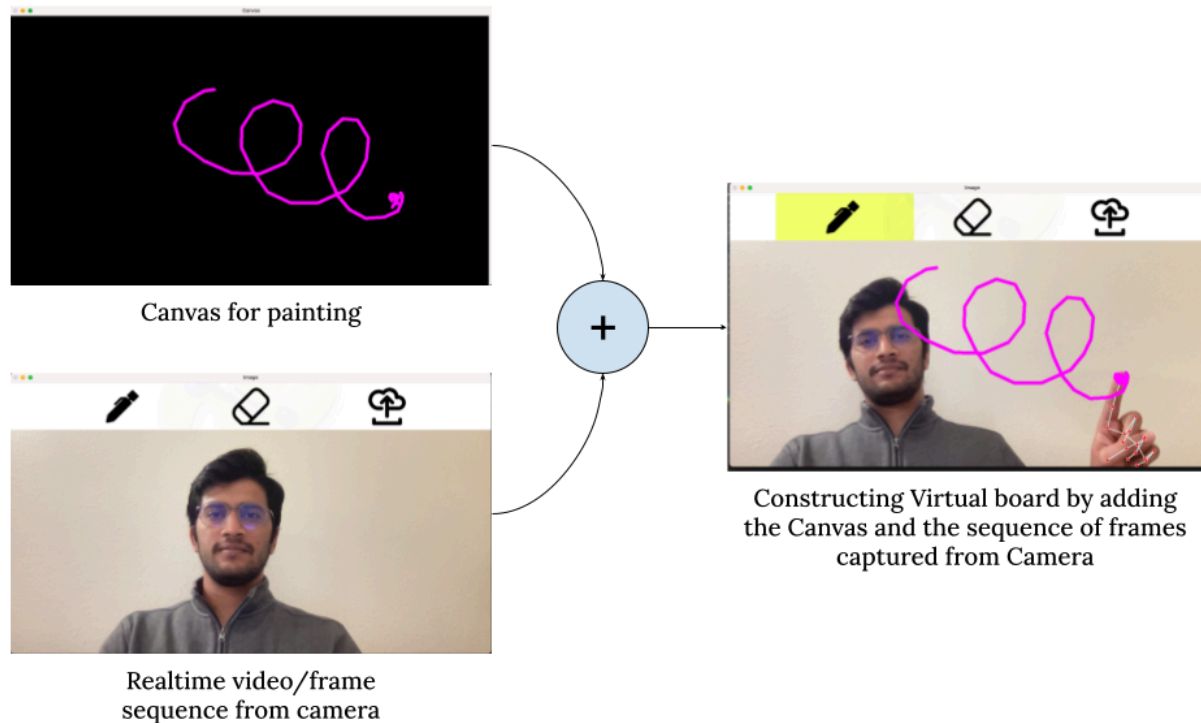


Figure 9 Virtual board painting via tracking index finger

Optical Character Recognition

After a user completes drawing their gesture/painting the next step would be to recognize the numerical digit drawn. For this purpose the *Google Tesseract* library has been used since it has a very quick (< 1 sec) turnaround time. Although the Tesseract API has been trained to recognize printed texts, its performance on hand written numerals is quite appreciable. But before the canvas painted by the user is sent to the Tesseract process we do certain pre-processing tasks on the painting so that it's easier for tesseract to perform and identify characters. The image pre-processing tasks include contour extraction of the drawn painting and smoothing the contour boundary to reduce the effects of freehand drawing.

The extracted contour is padded with borders and a new image is reconstructed as shown in the above figure. This image is converted to a byte stream and sent as an input to the *Tesseract API* for optical character recognition. Below is a code snippet for the following tasks-

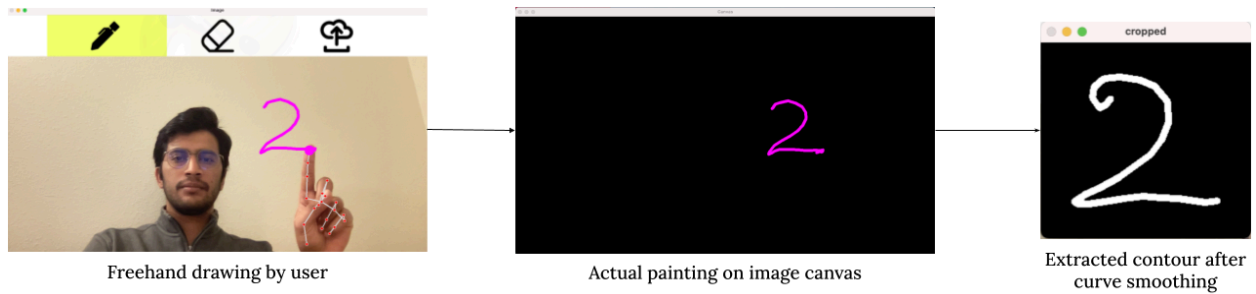


Figure 10 Image pre-processing and contour extraction

```
# text detection
def contours_text(self, orig, image_canvas, contours):
    output_text = ""
    for cnt in contours:
        x, y, w, h = cv2.boundingRect(cnt)

        # Drawing a rectangle on copied image
        rect = cv2.rectangle(image_canvas, (x, y), (x + w, y + h), (0, 255, 255), 2)

        cv2.imshow('cnt', rect)
        cv2.waitKey()

        # Cropping the text block for giving input to OCR
        cropped = orig[y:y + h, x:x + w]
        cropped = cv2.copyMakeBorder(
            cropped, 40, 40, 40, 40, cv2.BORDER_CONSTANT, value=[0, 0, 0])

        cv2.imshow('cropped', cropped)

        text = pytesseract.image_to_string(
            cropped, config=(-c tessedit
                            "_char_whitelist=0123456789"
                            " --psm 10"
                            "-l osd"
                            " "))

        print(f"Contour Text detected as: {text}")
        output_text = output_text + text
    return output_text
```

Figure 11 Code snippet to make Tesseract API call for OCR

The detected character by *Tesseract* is then sent back to the Unity based C# backend so as to initialize the next important process of summoning creatures.

Unity Scene Setup

Scenes are where you work with content in Unity. They are assets that contain all or part of a game or application. For example, you might build a simple game in a single scene, while for a more complex game, you might use one scene per level, each with its own environments, characters, obstacles, decorations, and UI. When you create a new project and open it for the first time, Unity opens a sample scene that contains only a Camera and a Light. For the purpose of this game we deal only with one single scene named `DemoScene_FPS.unity`. The three major components in the scene include the following-



Figure 12 Unity scene setup for the virtual world

1. Environment

The environment component was created to match the theme of the game, set in a virtual world. So, elements like barrels, hut, church, well, house, trees etc. were included for the medieval/fantasy atmosphere. Every single component in the scene be it the trees, shrubs, houses, or the mountains are contained within the Environment. The Environment component contains the definition for the entire layout and terrain of the game.

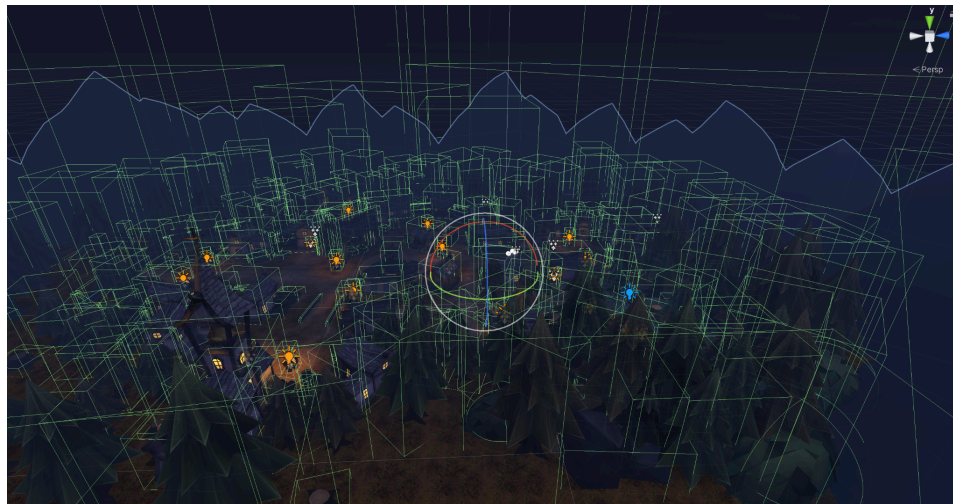


Figure 13 Environment component in Scene

2. Main Camera

Cameras are the devices that capture and display the world to the player. By customizing and manipulating cameras, we make the presentation of our game truly unique. They can be set to render in any order, at any place on the screen, or only certain parts of the

screen. Since we needed to have a virtual world experience, the best position for the camera would be to have it placed on the eye level of the user to have a first-person view.

MouseLook A *MouseLook* component is added to it for enabling users to look around in the virtual world. The hardware pointer is locked to the center of the view by setting the property `Cursor.lockState` as `CursorLockMode.Locked`.

3. Wand

Our *MainCamera* gameobject also contains the *Wand* gameobject so that it always stays in sight of the user at a fixed position. Wand is of the essence in our game as for the user the *spell-casting* should seemingly initiate from it. Lightning effects are added to the wand for the purpose, queued at a spell cast.

Aside from adding the visual element to the game, it also serves to hold the *Spell* component that performs functions like the instantiation of creatures and various commands to the creatures, once the argument (creature type or command type) is known.

4. Canvas

Canvas is the area that all UI elements should be inside. The Canvas is a Game Object with a Canvas component on it, and all UI elements must be children of such a Canvas.

Picture in Picture The Canvas gameobject holds the component to enable Picture in Picture (PIP) mode discussed already. As soon as the gesture is detected the image rendered onto the screen is disabled and the PIP component disappears from the screen when the user is not casting a spell.

```
// Disable the picture-in-picture mode once a new monster is summoned
RawImage image = GameObject.FindGameObjectWithTag("PIP").GetComponent<RawImage>();
image.enabled = false;
```

Figure 14 Code snippet to disable Picture in Picture mode within Scene

Crosshair A sight placed on the canvas used to aim and provide a point of reference to the player when looking and interacting with the game world. Although it is just a static object and doesn't react to any game events, it serves the purpose of letting the user know what and where they are going to shoot.



Figure 15 Crosshair is the orange dot at the center of the screen

Unity Raycast

A ray is an invisible line from Point A to Point B in the game. The Raycast function in Unity projects a Ray into the scene, returning a boolean value if a target was successfully hit or not. When this happens, information about the hit point, such as the distance, position, or a reference to the object's Transform, can be stored in a RaycastHit variable for further use. We use it for instantiating gameObjects on the position of hit (normal to it).

```
// Fire a raycast from the wand in the direction of camera with the range defined
bool rayIntersection = Physics.Raycast( fpsCam.transform.position, fpsCam.transform.forward, out hit, range);

// Debug.Log("Raycast hit location: " + hit.transform.name);
Debug.Log("Raycast hit point: " + hit.point);
```

Figure 16 Code snippet to initiate a Raycast within Scene



Figure 17 Path of a Raycast in Game

Monsters/Creatures

Monsters/Creatures is what makes the game interesting and fun! Each monster prefab is located in the Assets folder. A list of these monsters is created prior to starting the game. Based on the number detected via OpenCV, a particular monster from the list is summoned based on an index equal to the number detected by the OpenCV backend. These monsters are nothing but game objects in the context of unity, hence they could be easily instantiated or destroyed by calling specific methods. Each monster can be associated with a specific animation such that the monster keeps following those set of animator rules as long as they are alive. Since the monsters are game objects they could also have a Box Collider component defined where the mesh of this collider can be set up as the creator of the game wants.



Figure 18 Different monsters summoned in Game

```
// Method to summon a new monster
public void SpawnMonster()
{
    // summon a new monster from the monster list
    if( monsterList.Length != 0 )
    {
        GameObject monsterGO = Instantiate( monsterList[monsterIndex], hit.point, Quaternion.Euler(0, -hit.transform.rotation.y, 0) );
        monsterGO.tag = "monster";
        monsterGO.transform.LookAt(gameObject.transform);
    }

    // // Disable the picture-in-picture mode once a new monster is summoned
    // RawImage image = GameObject.FindGameObjectWithTag("PIP").GetComponent<RawImage>();
    // image.enabled = false;
}
```

Figure 19 `SpawnMonster()` method to create new monsters in scene

Particle System

The Particle System in Unity is a robust particle effect system where you can simulate moving liquids, smoke, clouds, flames, magic spells, and a whole slew of other effects. The Particle System has some base settings that we can configure. We also notice a large selection of tabs within the Particle System Inspector, called Modules. These Modules contain further settings for the Particle System.



Figure 20 Particle System simulating the magical wand in the game

For the purpose of this game we use the Particle system in three specific scenarios. First, to simulate the magical wand (as shown in the image above). Second, while summoning monsters and third while destroying monsters. The Particle System is also a game object and hence can be instantiated for the duration we want the animation to play and destroyed later on. The code snippet below gives an implementation idea for the same.

```
// Generate a game object to simulate the generation of monster
GameObject impactPS = Instantiate( destroyEffect, bestTarget.transform.position, Quaternion.Euler(0, -hit.transform.rotation.y, 0) );

// Destroy the game object after a specific delay
Destroy(impactPS, delay);

Destroy(bestTarget, delay);
```

Figure 21 Adding effects for monster creation through a short lived Particle System

Special Effects

Apart from the above modules and functionalities there are some other special effects defined in the context of the Unity game. One interesting aspect being the ability to destroy monsters. Everytime a Raycast is instantiated, we check internally if the hit point of the Raycast is close to one of the spawned monsters already in the game. If the new Raycast is close enough then we destroy the corresponding monster! Below figure is a visual explanation of the same.



Figure 22 Using Raycast system to destroy monster


```

public float GetClosestMonsterDistance(Vector3 rayHitPosition, GameObject[] instantiatedMonsterList, out GameObject bestTarget)
{
    bestTarget = null;
    float closestDistance = Mathf.Infinity;
    Debug.Log("GetClosestMonsterDistance ...");

    if(instantiatedMonsterList.Length > 0)
    {
        foreach (GameObject potentialTarget in instantiatedMonsterList)
        {
            // Debug.Log("Monster position: " + potentialTarget.transform.position);
            Debug.Log("Raycast hit point: " + rayHitPosition);

            Vector3 directionToTarget = potentialTarget.transform.position - rayHitPosition;

            float distanceToTarget = directionToTarget.magnitude;

            if (distanceToTarget < closestDistance)
            {
                closestDistance = distanceToTarget;
                bestTarget = potentialTarget;
            }
        }

        if(null != bestTarget)
        {
            Debug.Log("Target to be destroyed: " + bestTarget);
        }
    }

    return closestDistance;
}

```

Figure 23 Code snippet to destroy monster

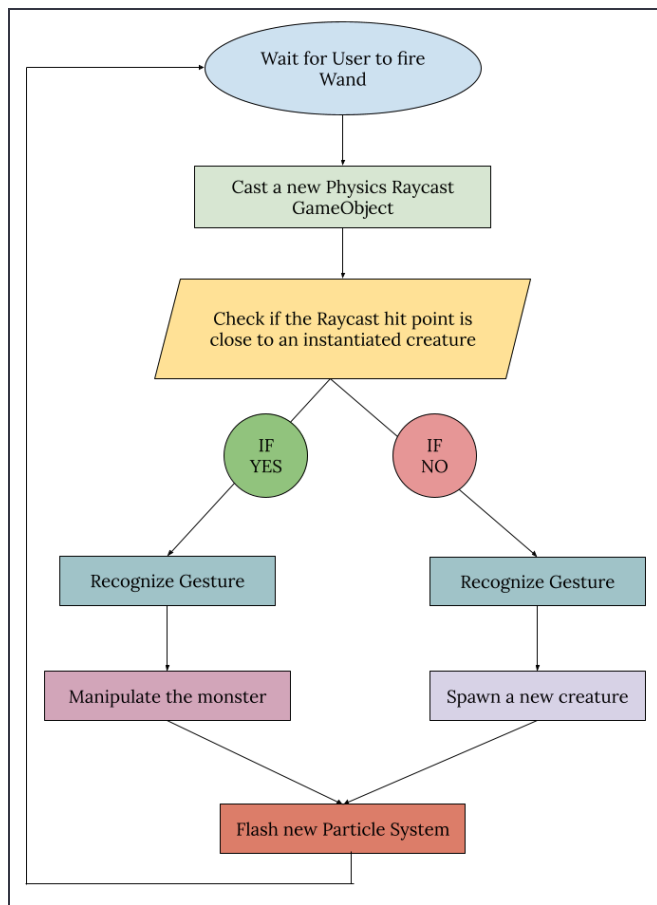


Figure 24 Workflow of the spell-cast operation from the wand. It shows the complete workflow of our spell-cast operation from the wand. So, the first decision we make is to check if the raycast is hitting an already summoned monster since in this case the spell/gesture would decide the command passed to the monster. Otherwise, the gesture from the user will be used to summon a new creature.

Demo

[Video link](#)

User Evaluation

To obtain feedback and evaluate our system, we invited 10 participants to try out our game and move around freely in the VR world, spawn a creature, give commands to it and to repeat the process as many times as they like. All the participants had experience of playing a first person game on a computer. A brief tutorial regarding all the functions of the system was provided before the task. We asked them to fill out a [short questionnaire](#) for subjective evaluation.

The statistical analysis is shown in **Figure 25**. We use the questionnaire to collect the subjective rating for the following questions with rating from 1 to 10-

Q1. How instinctive and natural it felt to draw out numbers in the Picture in Picture mode display?

[1-least, 10-most]

Q2. Did the commands always work on the creature? *[1-never, 10-always]*

Q3. How much learning time should be ideal for the game? *[1-minimal, 10-thorough]*

Q4. Faced with any jarring lags? *[1-none, 10-a lot]*

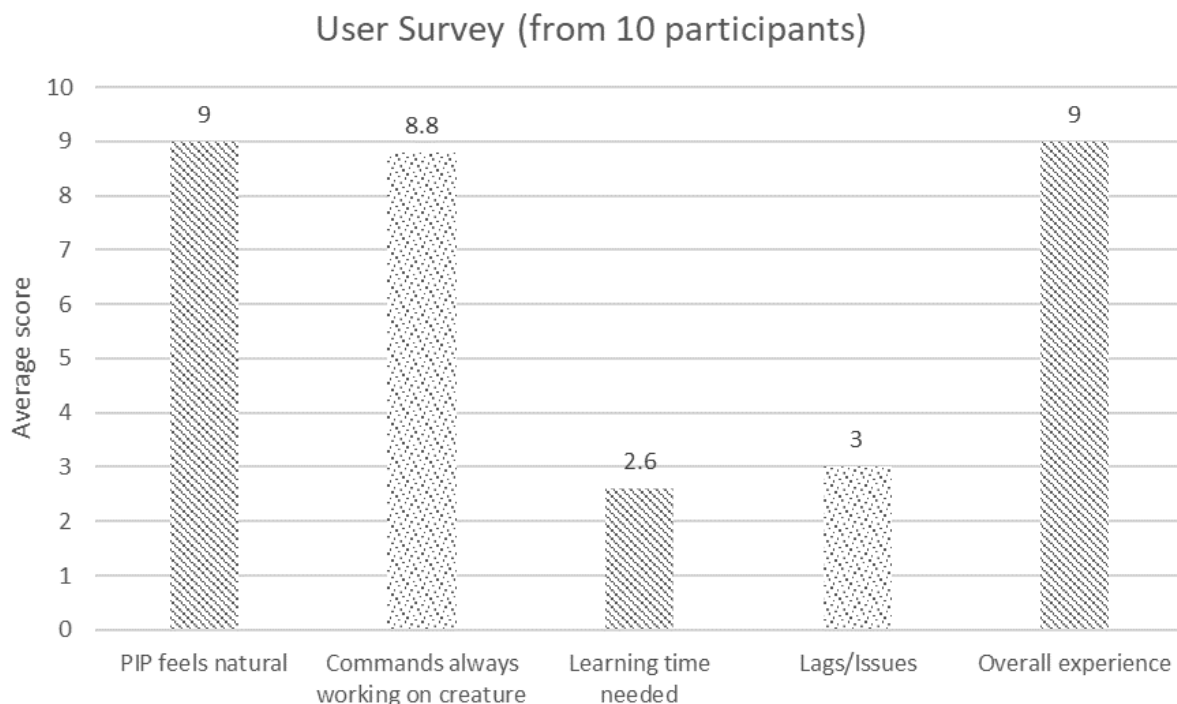


Figure 25 User Evaluation Summary

We are happy to report that with an average score of 9 from Q1, most of the participants found the interface for sketching to be natural which was a salient point during the development. Similarly, we got 8.8 for Q2 meaning the system was not facing hiccups during runs as observed by users.

Still, the participants felt that to use our system a user would need some learning and gave an average score of 2.6. We do realize that especially since the user needs to memorize the spell symbols, controls also might require briefing to the uninitiated. Q4 got an average score of 3 from the participants when asked about the lagging in the game. Lags might be interpreted as running heavy algorithms on a not so capable machine. The computer we used for the purpose (Apple M1 Mac 2020) was up to specs for our usage and didn't suffer from lags. Had the user given a high score for Q4, it would have pointed to a performance issue or an inefficient algorithm.

In addition some participants pointed out that there could be better ways the wand could be rendered on the screen. A hand model could be added to the scene holding the wand instead of showing a floating wand. Instead of PIP, there could be an interaction method where the wand is made to follow the user's index finger movement. This way we don't need to show the webcam feed on the screen which occludes game scenes and can even be considered intrusive to a user's privacy. Another participant suggested that we should explore our game in a multiplayer environment and let users cast spells on each other - stunning, scaling or teleporting them. Furthermore, the monsters can be equipped with AI using APIs like GPT-3 and openAI.

Conclusion and Future Work

A novel interaction system involving hand gestures collected by a webcam feed to be used in gaming controls was explored. The integration of our gesture recognition system into the VR world was effective (as shown by user survey) and can even be explored as another interaction method to be used in applications like twitch by streamers where the user needs multiple controls for gaming/steaming.

However, we felt that there can be more to add to our interaction system. We can explore sketching in 3D space to allow for a greater range of functions possible like spawn can be modified to draw out a 3D figure which is classified, using gestures like pinch/push for scaling/moving objects. Furthermore, as pointed out by the participants, the game can include a multiplayer mode and an improved wand, tracking and following the movement of the index finger while sketching instead of the PIP based interaction currently employed.

Contributions

Satyam was responsible for the majority of the unity app-side development for the Virtual Reality world. He developed the scenes, defined actions on user event triggers, added animated 3D models and stunning environments. He also contributed towards building socket-based communication between the two independent backend systems, and the picture in picture component of the application.

Shubham led the development of the gesture recognition and image processing module of the system. He developed the virtual canvas board and integrated a suite of APIs into the backend system for finger-tracking and OCR. On the Unity side of the game he contributed towards scripting some special effects into the game such as the ability of the user to destroy, transform, teleport and scale monsters.

GitHub Links for the Repo-

1. https://github.com/shubhamtalbar96/virtual_board
2. <https://github.com/SatyamA007/Spell-Training-in-VR>